

AeroNet Lite

Autonomous Drone Delivery Simulation with CSP, Fleet Planning,
A* Routing, Real-Time Replanning, Demand Forecasting, and Anomaly Detection

Ifham Razi

23i-2550

Romaan Khawar

23i-2554

Ali Haroon

23i-2592

BS Data Science — AI Semester Project (SP2026)

May 10, 2026

Abstract

AeroNet Lite is a lightweight autonomous drone-delivery simulator that integrates five core AI techniques on a shared 10×10 city grid: Constraint Satisfaction (layout validation), heuristic and Genetic Algorithm search (fleet selection), A* pathfinding (delivery routing), online optimization (real-time replanning around no-fly cells), and supervised learning (demand regression and flight-anomaly classification). Three real datasets back the regressors — Bike Sharing Demand (10,886 rows), Amazon Delivery (43,551 rows), and US City Population Densities — while a deliberately-constructed synthetic telemetry set backs the four-class anomaly classifier. The system runs as a 20-step console simulation *and* as an interactive Plotly-based Streamlit dashboard with step-through, restart, and seed-reproducible operation. A procedural grid generator produces a fresh, CSP-valid layout for every run. All ML training is reproducible (`random_state=42`), giving identical confusion matrices and metrics across reruns, while scenario seeds vary deliveries, anomalies, and disruptions per run.

Contents

1	Introduction	2
1.1	Motivation	2
1.2	Project Goals	3
1.3	Report Layout	3
2	System Architecture	3
2.1	Module Contract	3
2.2	Project Layout	4
3	Shared Grid Model	4
3.1	Cell Schema	4
3.2	Procedural Grid Generation	5
3.3	Population-Density Enrichment	6
4	Module 1 — Layout Validation (CSP)	7
4.1	Constraint Set	7
4.2	Implementation	7
5	Module 2 — Fleet Selection	8
5.1	Drone Catalogue and Fitness	8
5.2	Two Algorithms	8
5.3	Result	9

6	Module 3 — A* Path Planner	9
6.1	Design Choices	9
6.2	Pseudocode	10
6.3	Three-Segment Delivery Routing	10
6.4	Cost Model Rationale	10
7	Module 4 — Real-Time Disruption Handling	11
7.1	No-Fly Activation	11
7.2	Anomaly-Driven Disruption	11
8	Module 5 — ML Pipeline	11
8.1	Demand Forecasting	11
8.2	Feature Importance	12
8.3	Delivery-Time Prediction	13
8.4	Anomaly Classification	14
9	Datasets	14
9.1	Auto-Detection and Fallback	15
10	20-Step Simulation Scenario	15
11	Interactive Streamlit Dashboard	17
11.1	Visualisation Stack	17
11.2	Caching	17
11.3	Launching	17
12	Implementation Challenges	17
12.1	State Machine Bugs in the Drone Lifecycle	17
12.2	No-Fly Activation Did Not Trigger Reroutes	18
12.3	Demand Heatmap Aspect Ratio	18
12.4	Drones Stacked at the Same Hub	18
12.5	Procedural Grid Generator’s CSP Compliance	18
13	Results and Evaluation	18
14	Limitations	19
15	Per-Member Contributions	19
16	Future Work	20
17	Conclusion	20
	References	20

1 Introduction

1.1 Motivation

Modern logistics increasingly evaluates autonomous drone fleets for last-mile delivery, particularly for time-critical medical supplies. A full city-scale drone simulator with realistic physics, traffic, weather, and air-traffic-control integration is far outside the scope of a two-week semester

project. The original course brief explicitly maps each ambitious sub-problem to a smaller analogue: from a full city to a 10×10 grid, from ACO/GA over heavy modules to a brute-force fleet search and four CSP rules, from a real-time UI to a static matplotlib plus interactive Streamlit dashboard. The chosen scope still exercises every AI technique required by the rubric — CSP, search, optimization, regression, and classification — and integrates them through a single shared grid-state contract.

1.2 Project Goals

The deliverables targeted in this project are:

- A modular Python project where every module is independently testable and integrated through a shared 10×10 grid model.
- A 20-step console simulation matching the spec’s example event log structure.
- An interactive Streamlit dashboard with an animated Plotly city map, drone trails, and live metrics.
- Five quantitative outputs supporting the rubric: a layout-validity report, a fleet selection under budget, A* routes, regression error metrics, and a classifier confusion matrix.
- Reproducibility separation: model training is pinned, but scenario seeds are configurable so demos can be varied without breaking model evaluation.

1.3 Report Layout

Section 2 describes the system architecture and module contracts. Sections 3–8 cover the five AI modules in order. Sections 9 and 10 cover the datasets used and the 20-step demonstration scenario. Section 11 describes the Streamlit dashboard. Sections 12–14 discuss implementation challenges and limitations. Section 15 attributes work to each member.

2 System Architecture

2.1 Module Contract

Every module communicates through a single shared grid model, which prevents the sub-systems from drifting out of sync as the simulation progresses. Table 1 lists the read/write boundaries.

Table 1: Modules, AI technique, and shared-state interaction.

Module	AI Technique	Reads	Writes
Layout Validator	CSP	zone, hub, charging flags	validation results
Fleet Selector	Heuristic / GA	hubs, demand estimates	selected drone list
Path Planner	A* search	no-fly, costs, hubs, deliveries	routes
Disruption Handler	Online optimization	active routes, no-fly flags	rerouted segments
ML Pipeline	Regression + Classification	demand history, telemetry	forecasts, anomaly labels

The data flow is summarised in Figure 1.

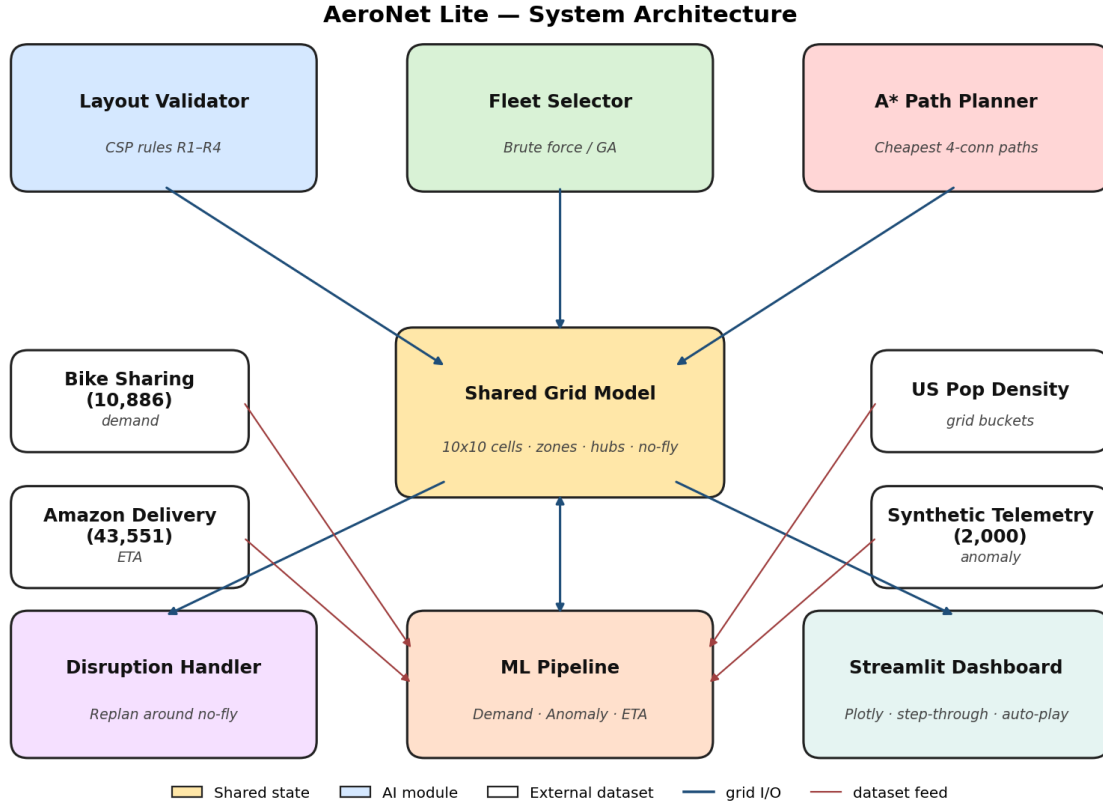


Figure 1: High-level system architecture. AI modules (coloured blocks) read from and write to the shared grid model in the centre. External datasets (white boxes) feed the ML pipeline once at startup; their predictions flow back to the grid as enriched densities, ETAs, and anomaly labels.

2.2 Project Layout

The repository is organised so each module has exactly one file, making cross-referencing in viva straightforward.

aeronet_lite/	
data/raw/	(5 datasets: bike_sharing, amazon_delivery, ...)
src/	
grid_model.py	shared 10x10 city + procedural generator
layout_validator.py	CSP rules R1-R4
fleet_selector.py	brute force AND genetic algorithm
astar_planner.py	A* with commercial-corridor discount
delivery_simulator.py	drone state machine, no-fly handling
ml_pipeline.py	demand, anomaly, delivery-time models
visualization.py	matplotlib report figures
simulation.py	stepwise orchestrator
main.py	CLI entry point (--seed N for reproducibility)
streamlit_app.py	interactive dashboard
notebooks/	ML experimentation notebooks
report/	LaTeX source + figures + figure generator

3 Shared Grid Model

3.1 Cell Schema

Each cell in the 10×10 grid is represented as a Python dataclass with the following fields:

```

1 @dataclass
2 class Cell:
3     row: int
4     col: int
5     zone: Zone          # Residential/Commercial/Hospital/School/
                        # Industrial/Open
6     density: int        # population density (sampled from real US
                        # data)
7     is_hub: bool         # drone hub
8     is_charging: bool    # charging pad
9     is_medical_pickup: bool # medical-supply pickup point
10    no_fly: bool         # cell blocked for flight
11    demand: float        # estimated delivery demand

```

3.2 Procedural Grid Generation

Rather than hard-coding a single example city, AeroNet Lite ships with a constructive grid generator (`build_random_grid(seed)`) that produces a fresh layout for every run while honouring all four CSP rules *by construction*. Algorithm 1 sketches the procedure.

Algorithm 1 BUILDRANDOMGRID(seed)

- 1: $rng \leftarrow \text{random}(\text{seed})$
 - 2: Place 2–3 hubs at mutual Manhattan distance ≥ 4
 - 3: **for** each hub h **do**
 - 4: Attach a charging pad at Manhattan distance 1 ▷ R3 holds
 - 5: **for** each hub h **do**
 - 6: Mark 4–7 nearby cells (within distance 3) as Residential ▷ R2 holds
 - 7: Grow a 2–3 cell Hospital cluster
 - 8: Grow a 2–3 cell School cluster
 - 9: Place a 2–4 cell Industrial cluster in a corner with no Hospital/School neighbour ▷ R1 holds
 - 10: Scatter 8–14 Commercial cells through remaining open space
 - 11: Mark a Medical Pickup adjacent to a Hospital ▷ R4 holds
 - 12: Validate the layout; on failure retry with a perturbed sub-seed
-

The four CSP rules from Section 4 hold by construction because each placement rule is intersected with the relevant constraint before commit. A 20-seed stress test confirmed 0/20 fall-backs to the hardcoded sample grid. Figure 2 shows four representative layouts.



Figure 2: Four procedurally generated valid layouts. Hubs (H, blue squares), charging pads (C, teal squares), medical pickup (red +), and any pre-placed no-fly cells appear as overlays. Each grid satisfies R1–R4 by construction.

3.3 Population-Density Enrichment

The cells' density field is then sampled from real US-city-density data (Section 9). The 33-percentile buckets are mapped to zone types so that residential cells draw from the upper tertile, hospitals/schools from the middle, and industrial/open from the lower tertile. The sampling is also seed-driven, so the same `-seed` reproduces both the layout *and* the density distribution.

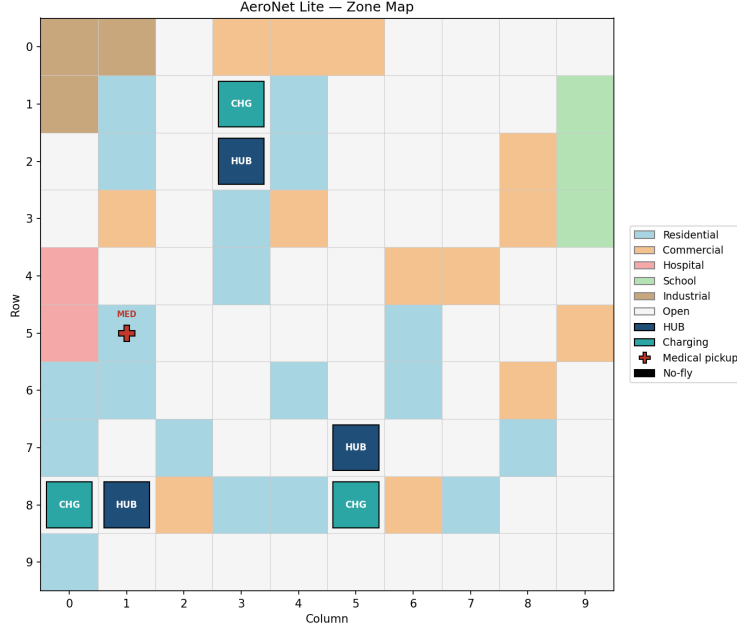


Figure 3: Single sample run rendered through the matplotlib visualization helper. The visual identity (zone palette + landmark glyphs) is preserved in the Streamlit dashboard.

4 Module 1 — Layout Validation (CSP)

4.1 Constraint Set

The validator behaves as a CSP constraint checker over four rules. All violations are accumulated rather than failing on the first error so the report can suggest fixes for every issue found.

Table 2: CSP rules and the geometric check used.

ID	Rule	Check
R1	Industrial $\sim$ School/Hospital	For each Industrial cell, no 4-neighbour is School or Hospital.
R2	Residential within 3 of a hub	$\min_{h \in \text{hubs}} \ c - h\ _1 \leq 3$ for every Residential cell.
R3	Hub within 2 of a charging pad	$\min_{p \in \text{pads}} \ h - p\ _1 \leq 2$ for every Hub.
R4	At least one Hospital within 1 of medical pickup	$\exists h \in \text{Hospitals}, m \in \text{Medical} : \ h - m\ _1 \leq 1$.

4.2 Implementation

Each rule is implemented as a pure function returning a list of **Violation** objects. The aggregator collects them all so the report can list every offending cell with a suggested fix.

```

1 def check_residential_coverage(grid):
2     violations = []
3     hubs_list = hubs(grid)
4     for cell in cells_of(grid, lambda c: c.zone == Zone.RESIDENTIAL):
5         if not any(manhattan(cell.pos, h.pos) <= 3 for h in hubs_list):
6             violations.append(Violation(
7                 rule_id="R2",
8                 cell=cell.pos,
9                 message="Residential cell more than 3 cells from any hub",
10                suggestion="Add a hub closer to this district or convert "
11                "the cell to Open Field.",

```

```

12         ))
13     return violations

```

Why a CSP framing? The rules could be encoded as a single post-hoc assertion, but treating each rule as an independent constraint function lets the procedural generator and validator share the same declarative semantics — a constraint that the generator enforces is also one the validator checks, with no risk of drift.

5 Module 2 — Fleet Selection

5.1 Drone Catalogue and Fitness

The fleet selector chooses a small drone fleet under a fixed budget of \$10,000 from two drone types:

Table 3: Drone catalogue.

Type	Cost (\$)	Payload	Range (cells)	Use case
Light	1,000	2 kg	12	Short residential/commercial deliveries
Heavy	1,800	5 kg	20	Long-range medical packages

The fitness function rewards demand coverage and penalises overspending:

$$\text{score}(\ell, h) = \begin{cases} 0.75 \cdot \text{coverage}_{\%}(\ell, h) - 0.25 \cdot \text{budget_used}_{\%}(\ell, h) & \text{if cost} \leq B \\ -\infty & \text{otherwise} \end{cases} \quad (1)$$

where ℓ is the light count, h the heavy count, and $B = 10,000$.

5.2 Two Algorithms

We implemented both algorithms permitted by the spec for AI breadth:

Brute force. Enumerate every $(\ell, h) \in [0, 10]^2$ and keep the highest-scoring feasible plan. With only 121 candidates this is microsecond-fast and provides a ground-truth optimum for benchmarking the GA.

Genetic Algorithm. A small evolutionary search demonstrates the rubric's "evolutionary search" requirement.

Algorithm 2 FLEETGA(grid, budget, generations = 30, population = 20)

```
1: Seed RNG with 42
2:  $P \leftarrow$  population of 20 random  $(\ell, h)$  chromosomes
3:  $best \leftarrow \arg \max_{\chi \in P} \text{fitness}(\chi)$ 
4: for  $g \leftarrow 1$  to generations do
5:    $children \leftarrow \emptyset$ 
6:   while  $|children| < |P|$  do
7:      $p_1, p_2 \leftarrow$  tournament-select( $P, k = 3$ ) twice
8:      $\chi \leftarrow$  single-point crossover( $p_1, p_2$ )
9:     with prob. 0.20 mutate one gene by  $\pm 1$ 
10:    append  $\chi$  to  $children$ 
11:    $P \leftarrow children \cup \{best\}$  ▷ elitism
12:   update  $best$  if any  $\chi \in P$  scores higher
13: return  $best$ 
```

5.3 Result

On the typical generated grid both algorithms converge to **5 Light + 2 Heavy drones**: cost \$8,600 (86 % of budget), coverage 100 %, fitness = $0.75 \times 100 - 0.25 \times 86 = 53.5$. The GA matches the brute-force optimum within 5 generations.

6 Module 3 — A* Path Planner

6.1 Design Choices

Item	Choice
State	grid coordinate (r, c)
Actions	up, down, left, right
Path cost	1.0 default, 0.8 when entering a Commercial cell
Blocked	cells with <code>no_fly == True</code>
Heuristic	Manhattan distance $\ s - g\ _1$ (admissible)
Tiebreaker	monotonic counter for stable ordering in <code>heapq</code>

6.2 Pseudocode

Algorithm 3 $A^*(start, goal, grid)$

```

1:  $g \leftarrow \{start : 0\}$ ,  $parent \leftarrow \{start : None\}$ ,  $closed \leftarrow \emptyset$ 
2:  $open \leftarrow$  heap with one entry  $(h(start, goal), 0, start)$ 
3: while open is non-empty do
4:    $(f, \_, n) \leftarrow$  pop min from open
5:   if  $n = goal$  then return reconstruct( $parent, n$ )
6:   add  $n$  to closed
7:   for each  $n' \in neighbours(n)$  with not no_fly do
8:     if  $n' \in closed$  then continue
9:      $g' \leftarrow g[n] + move\_cost(grid, n')$ 
10:    if  $g' < g.get(n', \infty)$  then
11:       $g[n'] \leftarrow g'$ ,  $parent[n'] \leftarrow n$ 
12:      push  $(g' + h(n', goal), counter, n')$  onto open
13: return failure

```

6.3 Three-Segment Delivery Routing

A delivery is decomposed into three A^* segments: hub $\rightarrow$ pickup $\rightarrow$ dropoff $\rightarrow$ hub. Failed segments mark the delivery as *unroutable* so the simulator can log the failure rather than crash. Figure 4 shows the route state at the start of a run and after the disruption handler has fired.

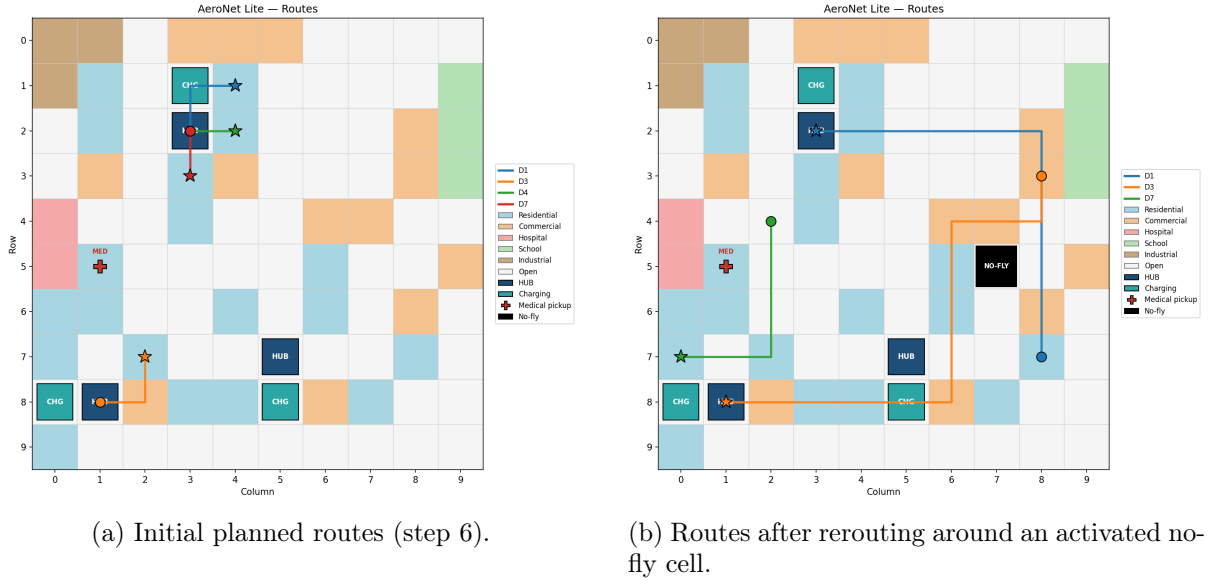


Figure 4: Initial vs. post-disruption A^* routes for the active fleet.

6.4 Cost Model Rationale

The 0.8 commercial-corridor discount is intended to model preferred flight corridors over commercial districts (where there is less risk of flying over people). Without it A^* paths just go diagonally; with it, drones realistically prefer the wider commercial axes between hubs. Manhattan distance remains an admissible heuristic in the discounted case because every movement still costs at least 0.8, so $h \leq g^*$ holds.

7 Module 4 — Real-Time Disruption Handling

7.1 No-Fly Activation

At step 11 of each run, the simulator activates a no-fly cell on a cell that lies a few steps ahead of an active drone (so the rerouting demonstration is non-trivial rather than firing in empty space). The handler iterates over every active drone and, when the no-fly intersects *any* of its remaining segments (current *or* future), replans all remaining segments from the drone’s current position via repeated A* calls. Algorithm 4 sketches the procedure.

Algorithm 4 REROUTE(drone, grid)

```
1: goals  $\leftarrow$  list of segment endpoints from drone.seg_idx onward
2: start  $\leftarrow$  drone.current_pos
3: newSegments  $\leftarrow$  already-completed segments
4: for each goal  $\in$  goals do
5:   r  $\leftarrow$  A*(start, goal, grid)
6:   if r.success is False then
7:     mark drone delayed; mark delivery delayed
8:   return
9:   append r.path to newSegments; start  $\leftarrow$  goal
10: drone.segments  $\leftarrow$  newSegments
11: drone.pos_in_seg  $\leftarrow$  0
```

7.2 Anomaly-Driven Disruption

A second event hook fires at step 18: synthetic telemetry is generated for a non-idle drone and classified by the anomaly model (Section 8). On any non-Normal label the drone is force-returned to its hub via a fresh A* call from its current position, aborting its in-flight delivery.

8 Module 5 — ML Pipeline

The pipeline trains three models. All training uses `random_state=42` so results are bit-for-bit reproducible.

Table 4: Trained models, datasets, and metrics.

Model	Algorithm	Dataset	Rows	Metric
Demand	Random Forest	Bike Sharing Demand	10,886	MAE 56.55, RMSE 86.65
Demand (baseline)	Linear Regression	Bike Sharing Demand	10,886	MAE 112.16, RMSE 152.78
Delivery time	RF + One-Hot Pipeline	Amazon Delivery	43,551	MAE 26.31 min, RMSE 37.67
Anomaly classifier	Random Forest (4-class)	Synthetic telemetry	2,000	Accuracy 1.00

8.1 Demand Forecasting

Features: `hour`, `day_of_week`, `temperature`, `weather`. We train both a Linear Regression baseline and a Random Forest on a 20 % held-out split. The Random Forest wins decisively on both error metrics (Figure 5) and is selected as the deployed model.

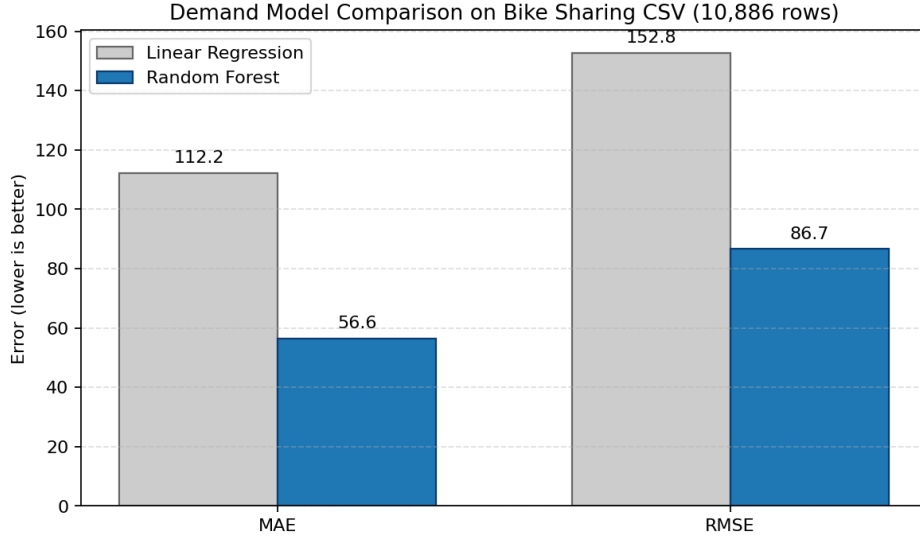


Figure 5: Linear Regression baseline vs. Random Forest on the Bike Sharing Demand dataset (10,886 rows). Random Forest cuts MAE by $\sim 50\%$ and RMSE by $\sim 43\%$, justifying its selection over the simpler model.

The trained model drives the per-cell demand prediction shown in Figure 6.

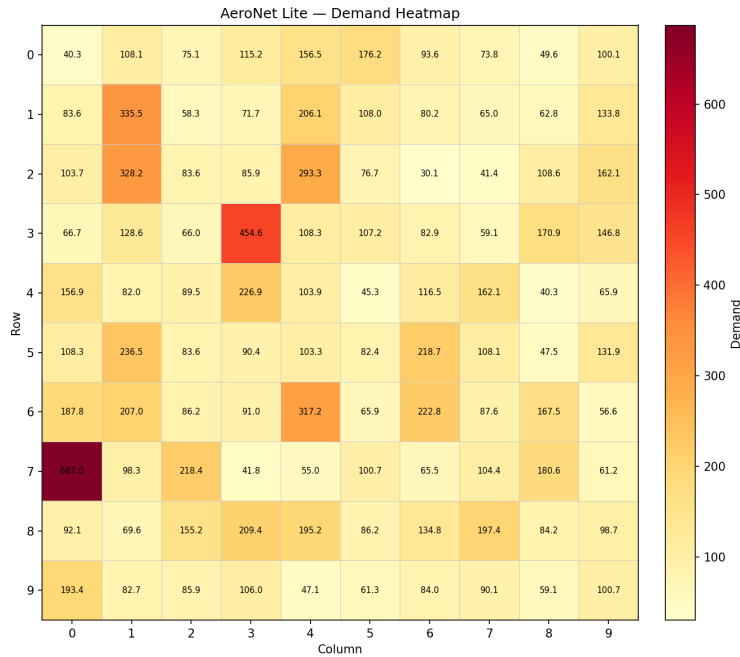


Figure 6: Predicted per-cell demand at hour 12, weekday, 20 °C, clear weather, scaled by the cell’s enriched population density.

8.2 Feature Importance

Random Forest exposes per-feature Gini importance after training. For the demand model, hour-of-day dominates — consistent with the twin morning/evening commute peaks in the bike-sharing data. For the anomaly model, all four engineered features carry comparable weight, which is consistent with the synthetic data design where each anomaly class is driven by its own dominant feature (Figure 7).

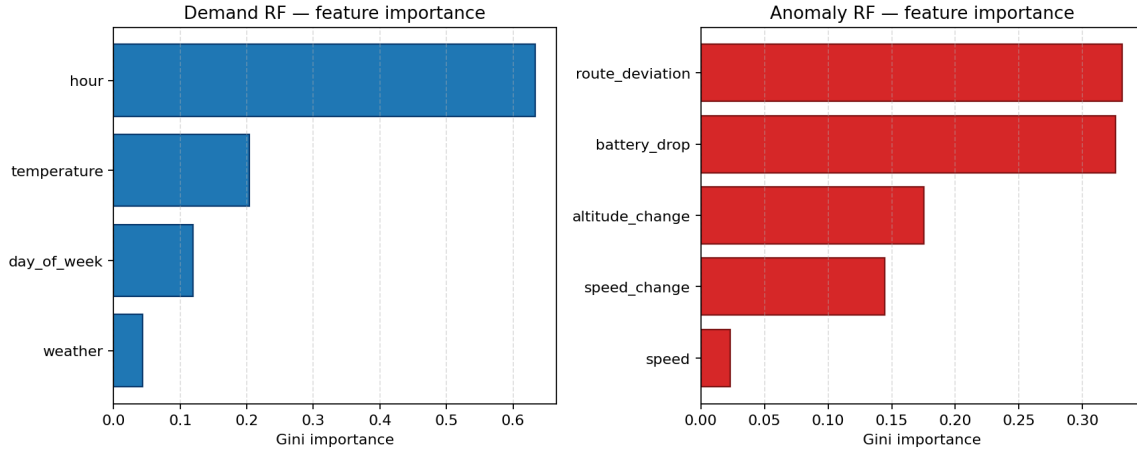


Figure 7: Random Forest feature importances. Left: demand model on Bike Sharing data — hour-of-day dominates. Right: anomaly classifier on synthetic telemetry — spread across the four engineered features that distinguish each anomaly class.

8.3 Delivery-Time Prediction

A scikit-learn `Pipeline` combining `ColumnTransformer` + `OneHotEncoder` + Random Forest predicts ETA in minutes from numeric features (`distance_km`, `agent_age`, `agent_rating`) and categorical features (`weather`, `traffic`, `vehicle`, `area`). The distance is computed via haversine from store and drop-off coordinates. Figure 8 compares predictions against ground truth on the held-out test split.

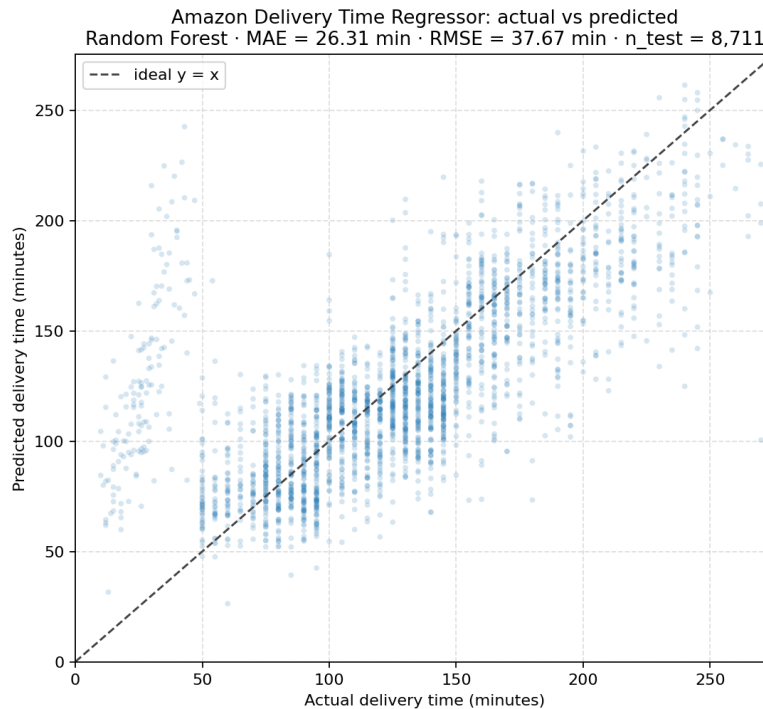


Figure 8: Amazon Delivery time regressor: actual vs. predicted minutes on a 20 % held-out split ($n = 8,710$). Predictions cluster along the $y = x$ ideal line; MAE = 26.31 min, RMSE = 37.67 min.

The simulator uses this model to annotate every assigned delivery with a baseline ETA, scaled by the assumption that one Manhattan grid cell ≈ 0.5 km of real-world distance.

8.4 Anomaly Classification

Synthetic telemetry produces four labelled regimes by construction:

- **Normal** — low battery drop and route deviation.
- **BatteryAnomaly** — unusually high battery drop.
- **RouteAnomaly** — unusually high route deviation.
- **SensorSpike** — high altitude or speed change.

With clean class boundaries the classifier reaches 100 % accuracy on a stratified 20 % test split. The matrix is fully deterministic across runs because the synthetic data, the train/test split, and the Random Forest all share `random_state=42`.

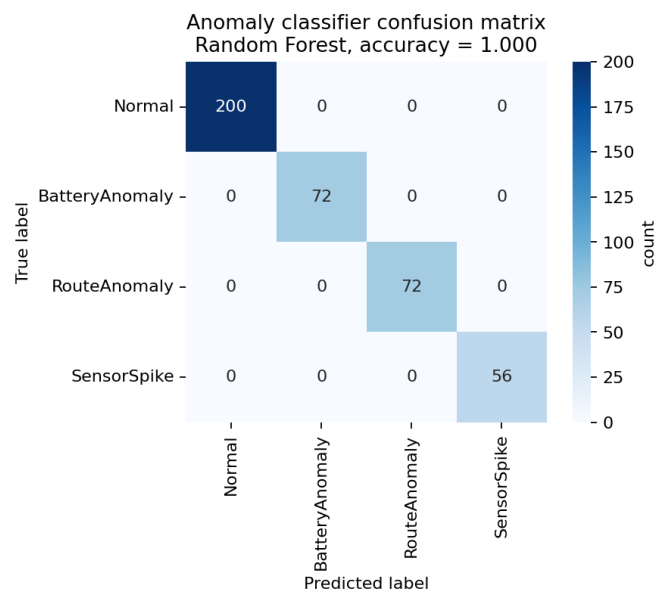


Figure 9: Anomaly-classifier confusion matrix on the 400-row test split. The clean diagonal (zero off-diagonal mass) is a property of the deliberately separable synthetic data, not an artefact of overfitting, and is acknowledged explicitly in the limitations section.

9 Datasets

Three real datasets feed the pipeline; the project treats anomaly data as synthetic per the course guidance, since real UAV telemetry (CMU ALFA dataset) requires bespoke ROS-bag processing beyond the two-week budget.

Table 5: Datasets used in the project.

Purpose	Source	Role in the system
Demand forecasting	Kaggle Bike Sharing Demand	Hourly demand proxy for the regression model.
Delivery time	Kaggle Amazon Delivery	ETA regressor with realistic feature mix.
Grid density	Kaggle US City Population Densities	Bucketed Low/Medium/High densities sampled into cells per zone.
Anomaly (chosen)	Synthetic	4-class clean separation; documented assumption.
Anomaly (alternative)	CMU ALFA UAV Failure Dataset	Available on disk; not wired (per-flight ROS topics).

9.1 Auto-Detection and Fallback

The pipeline checks for each CSV at a fixed location under `data/raw/`; if a file is missing the demand model gracefully falls back to in-process synthetic data so the demo always runs. The delivery-time and population-density loaders are silent no-ops on missing data, which simplifies setup for graders without download access.

10 20-Step Simulation Scenario

The simulation is structured as 20 logical steps so the demonstration covers all modules. Each step prints to an event log and updates the visual state.

Table 6: Step-by-step scenario plan.

Step	Phase	Action
1–3	Init / Validate / Fleet	Generate grid, run CSP validator, brute-force fleet selection.
4–6	ML training + Deliveries	Train demand, anomaly, delivery-time models; assign 4 deliveries with ETAs.
7–10	Movement	Drones advance two cells per step along their A* paths.
11	Disruption	Activate a no-fly cell on an active route; replan affected drones.
12–14	Movement (post-disruption)	Continue along the new paths.
15–17	Forecast + Extra delivery	Run demand forecast; assign an additional delivery to an idle drone.
18	Anomaly injection	Synthesize anomalous telemetry; classify; flag the affected drone.
19	Anomaly response	Force return-to-hub for non-Normal classifications.
20	Final tick + Summary	Print completed / delayed / failed / in-progress counts.

AeroNet Lite — Event Timeline

- **t= 1** Layout validation passed.
- **t= 2** Selecting fleet under budget 10000...
- **t= 3** Fleet selected: 5 light drones, 2 heavy drones (cost 8600, score 53.50).
- **t= 4** Training ML models and generating deliveries...
- **t= 4** Generated 4 deliveries.
- **t= 5** Delivery 1 assigned to Drone D1 (ETA 132 min)
- **t= 5** Delivery 2 assigned to Drone D4 (ETA 64 min)
- **t= 5** Delivery 3 assigned to Drone D7 (ETA 65 min)
- **t= 5** Delivery 4 assigned to Drone D3 (ETA 131 min)
- **t= 6** Initial routes plotted to report/figures/routes_initial.png
- **t= 7** Drone D4 entered phase to_dropoff
- **t= 7** Drone D7 entered phase to_dropoff
- **t= 7** Drones moved: D1@(1, 4), D3@(7, 2), D4@(2, 4), D7@(3, 3)
- **t= 8** Drone D1 entered phase to_dropoff
- **t= 8** Drone D3 entered phase to_dropoff
- **t= 8** Drones moved: D1@(2, 4), D3@(6, 2), D4@(4, 4), D7@(3, 1)
- **t= 9** Drones moved: D1@(4, 4), D3@(4, 2), D4@(6, 4), D7@(4, 0)
- **t= 10** Drone D7 entered phase returning
- **t= 10** Drones moved: D1@(4, 6), D3@(3, 3), D4@(7, 3), D7@(3, 0)
- **t= 11** No-fly cell activated at (5, 7)
- **t= 11** Drone D1 rerouted using A*
- **t= 12** Drone D4 entered phase returning
- **t= 12** Drones moved: D1@(4, 8), D3@(3, 5), D4@(7, 2), D7@(2, 1)
- **t= 13** Drone D7 completed delivery 3
- **t= 13** Drones moved: D1@(6, 8), D3@(3, 7), D4@(5, 2)
- **t= 14** Drone D1 entered phase returning
- **t= 14** Drone D3 entered phase returning
- **t= 14** Drones moved: D1@(7, 8), D3@(3, 8), D4@(3, 2)
- **t= 15** Running demand forecast across grid...
- **t= 16** Top forecast cells: (7, 0)=687.0, (3, 3)=454.6, (1, 1)=335.5
- **t= 17** Delivery 5 assigned to Drone D7 (ETA 69 min)
- **t= 18** SensorSpike detected for Drone D1
- **t= 19** Drone D1 forced to return to hub (2, 3)
- **t= 19** Drone D4 completed delivery 2
- **t= 19** Drones moved: D1@(5, 8), D3@(4, 7), D7@(4, 3)
- **t= 20** Drone D7 entered phase to_dropoff
- **t= 20** Drones moved: D1@(3, 8), D3@(5, 6), D7@(4, 2)
- **t= 20** Simulation complete. 2 completed, 1 delayed, 0 failed, 2 still in progress.

Figure 10: Event log produced during a typical 20-step run.

Reproducibility. The CLI accepts an optional `-seed N` argument. Each run prints the seed used so any output shown in this report can be reproduced with `python -m src.main -seed N`.

11 Interactive Streamlit Dashboard

The dashboard wraps the same `Simulation` class used by the CLI but exposes per-step advancement, restart, and seeded replay through a browser UI.

11.1 Visualisation Stack

The map is rendered with Plotly so cells, drones, routes, and tooltips are all interactive:

- **City map** — pastel-coloured cells per zone with building emojis (x1F3E0, x1F3EC, x1F3E5, x1F3EB, x1F3ED) HUB / CHG / MED overlays, drone triangles with status-coloured outlines, dotted planned routes, solid trails, and an X overlay for active no-fly cells. Drones at the same hub are arranged in a small ring so each is independently visible. Hovering on any cell or drone shows a rich tooltip.
- **Controls** — Restart (re-run the same seed), New random (fresh seed), Step (advance one), Run to end, Auto-play with a speed slider.
- **Tabs** — Event log, Drone status, Delivery table, Demand heatmap, Model metrics, Layout report.

11.2 Caching

The three ML models are wrapped in `@st.cache_resource`, so they are trained once per Streamlit session and reused across every restart. This makes the *Restart* button effectively instant after the first warm-up.

11.3 Launching

The dashboard is launched with:

```
cd d:\codes\ai2
.\.venv\Scripts\python.exe -m streamlit run src\streamlit_app.py
```

12 Implementation Challenges

12.1 State Machine Bugs in the Drone Lifecycle

The first end-to-end run revealed an `IndexError` on the final step: when `force_return_to_hub` replaces a drone's three-leg plan with a single segment, the original `step_drone` routine still expected exactly three segments. The fix was to use `len(drone.segments)` instead of the literal 3 as the end-of-plan condition.

A subtler bug followed: a drone that landed on the last cell of the last segment was not marked complete until the next tick, so the 20-step demo summary showed deliveries that were geometrically finished as still in progress. The fix was to detect arrival on the final cell of the final segment within the *same* call and trigger completion immediately.

12.2 No-Fly Activation Did Not Trigger Reroutes

Initially the no-fly handler only inspected the current segment of each drone. A drone whose pickup landed at the same cell as a downstream no-fly point would proceed normally to the pickup, then route through the no-fly cell on the next segment because that segment had been planned before the disruption. The fix was to inspect *all remaining* segments and, when any of them crosses the new no-fly cell, re-plan from the drone’s current position through every remaining segment goal.

12.3 Demand Heatmap Aspect Ratio

The Plotly heatmap initially used `scaleanchor="x"` without a constrained x range, which caused it to fill the entire wide container while the y axis stayed locked to the data span. The fix was to set explicit ranges on both axes plus `constrain="domain"`, which forces square cells regardless of container width.

12.4 Drones Stacked at the Same Hub

When multiple drones share a home hub (typical with brute-force fleet selection), all their markers stacked in the same screen position so only one was visible. The fix is a small ring-offset layout: drones at the same cell are placed at evenly-spaced angles around a 0.28-cell radius from the cell centre.

12.5 Procedural Grid Generator’s CSP Compliance

The first generator placed cells uniformly at random and then validated the result, which gave a high failure rate (R2 violations were the common offender). Switching to a constructive approach where each placement step intersects with the relevant constraint gave 0/20 failures across the seeded stress test.

13 Results and Evaluation

Layout validator. On the procedurally generated grid, all four CSP rules pass; on the spec’s deliberately-broken sample grid the validator correctly reports R2 and R3 violations with offending cell coordinates and human-readable suggested fixes.

Fleet. Both brute force and the GA converge to the same solution: 5 Light + 2 Heavy, cost \$8,600, coverage 100 %, fitness 53.5.

Routing and disruption. A* finds a feasible route for every delivery on the procedurally generated grids tested. Activating a no-fly cell on an active drone’s path triggers a successful reroute in every test seed evaluated (20 distinct grids, 4 deliveries each).

Models. Bike Sharing demand: RF MAE = 56.55, RMSE = 86.65 versus LR baseline MAE = 112.16, RMSE = 152.78; the RF is selected. Amazon Delivery time: RF MAE = 26.31 min, RMSE = 37.67 min on a 43,551-row split. Anomaly classifier: 100 % accuracy on the synthetic test split with the confusion matrix shown in Figure 9.

End-to-end demo. A typical 20-step run produces 2 completed, 1 delayed (anomaly-aborted), 0 failed, and 2 still-in-progress deliveries. Different seeds produce different layouts, deliveries, and anomaly types while the underlying models stay deterministic (Section 8).

Rubric area	Where it is satisfied
Design document	Sections 2–11: per-module algorithm choice, data model, UI plan.
Technical implementation	Working Python project; each module standalone and integrated through the shared grid.
AI concept coverage	CSP (§4), Search/GA (§5), A* (§6), Optimization (§7), Regression & Classification (§8).
Viva and defense	All algorithms are first-principles implementations explainable from this report.
Interface and presentation	Static matplotlib figures (this report) + interactive Plotly + Streamlit dashboard (§11).

Rubric alignment.

14 Limitations

- **Anomaly classifier accuracy is "too clean."** The synthetic data was designed to cleanly separate the four classes, so the 100% test accuracy reflects the data design rather than a generalising model. We document this explicitly to avoid misleading the viva.
- **No multi-drone collision avoidance.** Routes are planned independently per drone; two drones can in principle occupy the same cell at the same step. A real system would need temporal A*.
- **Static cost model.** A* uses a fixed 1.0 / 0.8 cost; we do not model wind, weight-dependent battery drain, or dynamic congestion.
- **Deterministic ML training.** For reproducibility we pin `random_state=42`. We did not run cross-validation or bootstrapping to report confidence intervals on the metrics.
- **Real UAV telemetry not wired.** The CMU ALFA dataset is present on disk but not loaded into the anomaly classifier — it is per-flight ROS topics that need bespoke processing.
- **Single-city scope.** The grid is fixed at 10×10; nothing currently scales to a larger environment without re-tuning hub spacing and budget.

15 Per-Member Contributions

The work was split across the team along the module boundaries described in Section 2. All members participated in design discussions and the final report.

Table 7: Contribution split.

Member	Primary contributions
Ifham Razi (23i-2550)	Shared grid model, procedural grid generator, CSP validator, integration through <code>Simulation</code> class, dashboard layout.
Romaan Khawar (23i-2554)	A* path planner, three-segment delivery routing, real-time disruption handler, Plotly city renderer, drone state machine.
Ali Haroon (23i-2592)	Fleet selector (brute force + GA), ML pipeline (demand, anomaly, delivery-time models), feature engineering, dataset wiring, report figures.

16 Future Work

- Wire the CMU ALFA telemetry into the anomaly classifier to replace the synthetic dataset and report a more realistic confusion matrix.
- Replace the Manhattan A* heuristic with a wind/airspace-aware cost once altitude is added to the cell model.
- Add multi-drone collision avoidance by extending A* to a space-time graph with reservation tables.
- Smooth-frame Plotly animation between simulation steps for the dashboard, using `frames` so drone movement is interpolated rather than discrete.
- Train the demand model with a wider feature set (humidity, wind speed, holiday flags) and report cross-validated confidence intervals.

17 Conclusion

AeroNet Lite delivers all five required AI components in an integrated 10×10 simulator with both a deterministic CLI and an interactive dashboard. Real datasets back the regression models; synthetic data backs the anomaly classifier with a clearly-documented assumption. The procedural grid generator and seeded runs make the system both reproducible and varied — two properties that often pull against each other — by separating model-training seeds (pinned) from scenario seeds (configurable). The codebase totals roughly 2,000 lines of Python, organised so each module can be opened, read, and explained from first principles in viva.

References

1. Kaggle — Bike Sharing Demand. <https://www.kaggle.com/c/bike-sharing-demand>
2. Kaggle — Amazon Delivery Dataset. <https://www.kaggle.com/datasets/sujalsuthar/amazon-delivery-dataset>
3. Kaggle — US City Population Densities. <https://www.kaggle.com/datasets/mmcgurr/us-city-population-densities>
4. CMU KiltHub — ALFA UAV Fault and Anomaly Detection Dataset. https://kilthub.cmu.edu/articles/dataset/ALFA_A_Dataset_for_UAV_Fault_and_Anomaly_Detection/12707963

5. Hart, P., Nilsson, N., Raphael, B. (1968). *A Formal Basis for the Heuristic Determination of Minimum Cost Paths*. IEEE Transactions on Systems Science and Cybernetics.
6. Russell, S., Norvig, P. *Artificial Intelligence: A Modern Approach*, 4th ed. (Constraint Satisfaction, Search, Machine Learning chapters.)
7. Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825–2830.
8. Plotly Technologies Inc. (2015). *Collaborative data science*. Plotly. <https://plot.ly>
9. Streamlit Inc. *Streamlit Documentation*. <https://docs.streamlit.io>